COSMICFORGE TECH LIMITED

CF MEET — PHASE 2
Frontend Product Design & Requirements (PDR)

Implementation blueprint, complete screen behavior, integration architecture and endpoint-by-endpoint reference

Version 1.0 | Issued 25 October 2026 | Status: Developer handoff draft (product decisions flagged)

Scope: Meeting & Conference Platform, web first; responsive/mobile integration requirements. Backend authority: Phase 2 Backend Implementation Specification v1.1, 03 October 2026. API authority: checked-in Meeting Service OpenAPI v1.2.0 reviewed 25 October 2026.

Non-negotiable design rule. MeetingSpace is a persistent configuration; ConferenceSession is a scheduled/live occurrence; ConferenceParticipant is a logical identity; ParticipantSession is a single device connection. Never substitute these identifiers.

Document control	Value
Owner / audience	Frontend engineering, UX, QA, backend integrations, product and security
Primary implementation repository	CosmicforgeHealthnet/cfmeet — apps/web (and shared frontend modules)
Backend contract	packages/contracts/openapi/meeting-service.json — v1.2.0, 205 operations (153 v2 / 50 v1 / 2 probes)
Companion workbook	CFMeet_Phase2_Swagger_Endpoint_Implementation_Matrix_v1.0.xlsx — 205 searchable rows
Change control	Pin API commit / generated client when implementation begins; re-run contract parity at release
Important distinction	Backend v1.1 PDF is the requirements origin; later Swagger v1.2.0 is the exact endpoint naming authority

Contents and reading guide

- 1. Executive implementation direction
- 2. Requirements basis, scope and sources
- 3. Users, roles and permissions
- 4. Information architecture and frontend routes
- 5. Screen-by-screen product specifications
- 6. Complete user journeys and state transitions
- 7. API integration architecture and type contracts
- 8. Realtime, RTC and reliability design
- 9. Security, privacy, accessibility and responsive behavior
- 10. Feature-gated modes and conditional experiences
- 11. Acceptance criteria and end-to-end certification
- 12. Delivery plan, dependencies and decisions
- Appendix A. Every Meeting Service endpoint (205 operations)
- Appendix B. Swagger cross-domain boundaries and glossary

How to use this PDR. Implement sections 1–12 as behavior and acceptance authority. Use Appendix A or the companion spreadsheet to look up any endpoint by path, purpose, screen, request schema and error handling. OpenAPI remains the source for exact field definitions and HTTP responses.

1. Executive implementation direction

Build a complete Phase 2 meeting frontend that exposes the implemented meeting and conferencing domain as one coherent product: discover/create spaces, schedule occurrences, join safely, manage permissions and media, collaborate live, recover from failures, and retrieve authorized post-meeting outputs. The existing web client is a migration input, not the UX or integration blueprint to duplicate.

1.1 Required architecture decisions

- Use /api/v2 for all new meeting-domain workflows. Preserve v1 only where a migration plan or account authentication explicitly needs it. Do not send v2 IDs to legacy meeting/chat/media endpoints.
- Treat the JoinConferenceResponse as a canonical identity/permissions handoff. Store spaceId, conferenceId, participantId and participantSessionId distinctly; retain only the appropriate server-issued guest credential.
- Keep two connected state planes: REST/WS for authoritative meeting state; LiveKit for tracks/devices/media rendering. UI must display control-plane permissions and actual provider state consistently.
- Every destructive or privileged action requires role/policy-aware UI, user confirmation where appropriate, backend authorization, and reconciliation after mutation.
- Use per-environment capability flags and effective meeting policies. Hide or explain disabled recording, healthcare, webinar, intelligence and similar modules rather than promising functionality based on Swagger existence alone.

1.2 Release outcomes

Outcome	Observable completion signal
End-to-end core meeting	Owner schedules/hosts; guest requests admission; host admits; media works; attendee leaves; owner ends; RTC closes.
Link compatibility	Current and legacy identifiers resolve to the canonical space/conference; no identity inferred from link.
Full meeting control	Co-host, moderator, presenter, participant and viewer UX reflects backend permissions, including RTC grants.
Reliable state	WebSocket loss, refresh, missed events and media reconnect reconcile to canonical state; no duplicate participants.
Advanced collaboration	Chat, hands/reactions, polls, Q&A and breakouts are policy-gated and multi-browser tested.
Gated additions	Recording/artifacts, transcription, webinar and healthcare experiences available only when provider/flag/profile evidence supports them.

2. Requirements basis, scope and sources

SOURCE A — Backend specification (uploaded): “CF MEET — PHASE 2 Meeting & Conference Platform Backend Implementation Specification”, v1.1, 03 Oct 2026. Its Parts A–N and 14 functional sections define the server/product obligations, and explicitly exclude frontend implementation. Page references in this document use the printed PDF pages.

SOURCE B — Implemented meeting contract: https://github.com/CosmicforgeHealthnet/cfmeet/blob/main/packages/contracts/openapi/meeting-service.json (version 1.2.0; 205 HTTP operations). It supersedes the 2026-10-03 PDF when a draft endpoint path differs from the implemented route.

SOURCE C — Current implementation/integration evidence: services/meeting-service/src/routes/v2.rs, services/meeting-service/docs/PHASE2_MEETING_DOMAIN.md; apps/web/src/components/room/, apps/web/src/services/v2/, docs/api/README.md. These inform the migration risks and the decisions in this PDR.

2.1 Scope boundaries

Included in Phase 2 frontend	Excluded / separate delivery
Meeting dashboard, meeting spaces, occurrences and recurrence	Public PSTN/telephony calling and SMS
Account authentication and guest admission	Number provisioning / DCNs, Privacy Relay
Waiting, roles, device sessions, media, chat, collaboration	Full general-purpose messaging/E2EE device workflows (Part V)
Recording/consent/artifacts and specialized modes where enabled	Developer CPaaS onboarding, pricing and billing (Part VI+)
Quality and recoverable meeting UX	Creating proprietary SFU, provider operations or administrative migrations

2.2 Specification-versus-current-contract reconciliation

Original PDF draft wording	Implemented Swagger for frontend use
GET /meeting-spaces/{id}/occurrences	GET /api/v2/meeting-spaces/{id}/conferences
GET /meeting-spaces/{idOrCode}	GET /api/v2/meeting-spaces/{id} plus GET /api/v2/resolve/{identifier}
PATCH /conferences/{id}/media-policy (representative)	Concrete audio/video/screen-share-lock endpoints, spotlight, collaboration-policy and media-credential
Recording/artifact representative endpoint families	Use their actual /api/v2/conferences/{id}/recordings, /artifacts and /consents operations
Simple join sample in PDF	Use the full v1.2.0 JoinConferenceResponse with participantToken, rtcStatus, participantSession, realtime, mediaContext, permissions and policy

3. Users, roles and permissions

The seven built-in meeting roles are OWNER, HOST, CO_HOST, MODERATOR, PRESENTER, PARTICIPANT and VIEWER. The backend grants capabilities; the frontend renders afforded controls. A user might hold membership in a persistent space and a different role in an occurrence. The registered-account/guest distinction is identity, not a role. See backend specification pp. 9 and 28.

Persona	Key frontend needs	Authority model
Owner / organizer	Space settings, membership, recurrence, profiles, host tools, results and history	Owner is verified by server, not meeting code.
Host	Start/end, admit/deny, locks, moderate, create polls/breakouts/recordings if permitted	Action buttons rendered from backend role/policy and permissions.
Co-host	Delegated host tasks, manage waiting room, assist while host away	Never assume full ownership transfer capability.
Moderator	Roster/content controls within allowed scope	May not have global meeting configuration access.
Presenter / panelist	Publish AV/share, stage/backstage and webinar interactions	RTC grants can change when promoted/demoted.
Participant / guest	Join safely, media controls, chats, hand/polls/Q&A	Guest relies on scoped participant token; displayName not identity.
Viewer / attendee	Watch/listen, questions, reactions, possible role promotion	No audio/video publishing unless server re-grants.
Organization/healthcare context	Verified organization membership or appointment-based join	Never expose protected title/records before authorized.

3.1 Frontend permission evaluation

1. Read effective permissions/policy returned by the backend (never infer from client-selected role).
2. Hide forbidden actions for clarity, and optionally show disabled actions with policy explanation where useful.
3. Prevent UI actions while request is pending, but rely on the backend for every authorization decision.
4. React to permission changes by stopping forbidden tracks/controls, refreshing current participant, and updating the toolbar.
5. Test manipulated clients cannot gain host/viewer/grant privileges; frontend prevention alone is never a security control.

4. Information architecture and frontend routes

Screen / route (proposed)	Purpose	Backend integration
/login, /register, /verify, /forgot-password	Account onboarding / recovery	v1 auth; /users/me
/dashboard	Owned/member spaces, scheduled/active conferences	v2 meeting-spaces, me/organizations
/meetings/new	Wizard: type, title, timezone, security, scheduling	v2 meeting-spaces, recurrence, conferences
/meetings/{spaceId}	Space detail, upcoming/past conferences, member/invite list	v2 meeting-spaces/{id}, conferences, members, invitations
/meetings/{spaceId}/settings	Security, roles, recording/transcription, profile policies	v2 PATCH space and profile-policy
/join/{identifier}	Resolve any meeting code/id/link and pre-join	v2 resolve/{identifier}, conferences/{id}/pre-join
/conferences/{conferenceId}/waiting	Waiting decision and reconnect UI	v2 participants/me, realtime, join
/conferences/{conferenceId}/live	Live media room and all collaboration panels	v2 conference endpoint families; LiveKit SDK
/conferences/{conferenceId}/summary	Ended state, attendance, artifacts, recordings/transcripts	v2 conference + artifact/recording/transcript APIs
/webinars/{spaceId}/register	Conditional public registration	v2 registrations
/organizations/{id}/members	Authorized org management	v2 org member endpoints only if selected in product scope

Navigation and privacy. The live room route must use conferenceId; meetingCode is a navigational entry identifier, not a replacement for conferenceId. Do not show private meeting metadata before an authorized response permits it.

5. Screen-by-screen product specifications

F-01 Authentication and account recovery

UI and behavior: Login, registration, email verification, OAuth initiation/callback, password reset and profile account settings. Distinguish anonymous guest meeting entry from account sign-in. Preserve safe intended navigation after login.

API family: POST /api/v1/auth/login; /register; /refresh; /logout; /forgot-password; /reset-password; /verify-email; GET/PATCH /api/v1/users/me

Required states and edge cases: Success, invalid login, unverified account, expired reset token, rate-limit, network fail, logout across tabs.

Acceptance: User can complete each approved account path without losing a meeting invitation. No raw tokens in URLs/logs.

F-02 Dashboard, filters and meeting list

UI and behavior: List owned/member spaces, next occurrence, LIVE badge, actions (start, join, edit, invite, duplicate, archive where permitted), empty state, pagination. Use space identity for cards and conference identity for Start/Join.

API family: GET /api/v2/meeting-spaces; GET /api/v2/me/organizations; GET /api/v2/meeting-spaces/{id}/conferences

Required states and edge cases: Loading, empty, stale list, unauthorized org, archived, pagination, active/expired conference.

Acceptance: Cards open correct latest/live occurrence; no stale active status after event or refresh.

F-03 Create meeting / instant meeting wizard

UI and behavior: Select instant versus scheduled/reusable; title, description, IANA timezone, meeting type, capacities, access policy, waiting room, security profile, default role, capture policies and optional organization. Show policy help and preview; generate share link after persisted response.

API family: POST /api/v2/meeting-spaces; POST /api/v2/instant-meetings; POST /api/v2/meeting-spaces/{id}/conferences

Required states and edge cases: Validation, unsupported meeting type, over-ceiling policy, create pending, 409, rollback, accidental double submit.

Acceptance: Instant produces a LIVE conference; scheduled produces a distinct occurrence; no client-selected owner ID.

F-04 Space details, editing and lifecycle

UI and behavior: Show persistent configuration, upcoming/past sessions, action log/metadata available to caller, duplicate/archive/owner transfer; use versioned forms and confirm destructive operations.

API family: GET/PATCH /api/v2/meeting-spaces/{id}; /archive; /duplicate; /owner-transfer; GET /conferences

Required states and edge cases: Read-only summary vs authorized full detail, version conflict, owner-only actions, archived state.

Acceptance: Refetch on 409 rather than overwriting updates; legacy code continues resolving.

F-05 Schedule and recurring series

UI and behavior: Timezone-aware date/time picker, next occurrence preview, RFC 5545-compatible recurrence, exceptions to cancel/reschedule ONE occurrence, end series with past data intact; accessible daylight-saving handling.

API family: GET/PUT/DELETE /api/v2/meeting-spaces/{id}/recurrence; GET/PUT /recurrence/exceptions; POST /api/v2/conferences/{id}/reschedule

Required states and edge cases: DST gaps/overlaps, missing recurrence, past occurrence edit, stale exceptions, optimistic conflict.

Acceptance: All dates display in selected meeting timezone and participants' local time with labels; exceptions do not modify entire series.

F-06 Membership, invitations, and share

UI and behavior: Persistent member list, add/update/revoke within grant authority, invite by email, allowed roles, use limits, expiry, specific occurrence; show invite link/token once; revoke invitations and display metadata without secrets.

API family: GET/POST /api/v2/meeting-spaces/{id}/members; PATCH/DELETE /members/{memberId}; GET/POST /invitations; POST /invitations/{invitationId}/revoke; POST /api/v2/invitations/{token}/resolve

Required states and edge cases: Expired/revoked/exhausted invite, permission denied, no member access, token one-time display.

Acceptance: Invite cannot elevate authority or leak other members; copy link and accessible share actions work.

F-07 Link resolution and pre-join lobby

UI and behavior: Resolve current or legacy ID/code; select canonical conference; display permitted meeting details, title, organizer, privacy notices, joinability, device capabilities; preview camera; choose mic/camera/speaker; test devices; consent and passcode/invite requirements.

API family: GET /api/v2/resolve/{identifier}; GET /api/v2/conferences/{id}/pre-join; GET /consents/requirements; POST /consents

Required states and edge cases: Meeting not found, private title suppression, offline/permissions blocked, not started/ended, passcode required, guest name, consent pending.

Acceptance: Do not request RTC credential until join is authorized; never use displayName as identity.

F-08 Admission and waiting room - attendee

UI and behavior: After POST join, examine admissionState/rtcStatus rather than treating a 200 as permission to connect. Render waiting/denied/removed/banned/locked separate screens. Subscribe SELF_ONLY events; poll participants/me as fallback; only request/receive media credential when ADMITTED.

API family: POST /api/v2/conferences/{id}/join; GET /participants/me; POST /realtime/ticket; GET /events

Required states and edge cases: Waiting, host denied with safe message, invite expired, transient WS loss, admitted during sleep, join timeout.

Acceptance: No LiveKit connection while WAITING; after admit, get authorized RTC credential and connect once.

F-09 Host admission console

UI and behavior: Queue with request time/identity context allowed by policy; bulk admit/deny/all, optional reason/message; return admitted participant to waiting; indicate failures per action and update rosters.

API family: GET /api/v2/conferences/{id}/waiting-room; POST /waiting-room/admit, /deny; POST /participants/{participantId}/return-to-waiting

Required states and edge cases: Queue churn, idempotent retries, cross-meeting ID rejected, ability revoked, latency/replay.

Acceptance: Bulk actions have explicit confirmation where destructive, pending state and post-mutation reconciliation.

F-10 Live meeting core

UI and behavior: Media grid/active speaker, screen share stage, compact mobile view, participant count, connection state, meeting title/time, readable accessible controls, sidebar tabs. Distinguish media provider events vs conference authoritative events.

API family: GET /api/v2/conferences/{id}; /participants; /participants/me; /collaboration; /realtime/ticket; LiveKit room

Required states and edge cases: Permission grant change, RTC disconnected but WebSocket live, meeting ENDING, current session expired, stale roster.

Acceptance: Media starts only after credential; UI reacts to back-end end/remove/ban independent of local disconnect.

F-11 Personal audio/video and screen sharing

UI and behavior: Mic/camera toggles, device selection and levels, screen/window/tab selection, view-only state, publishing denied explanations. Apply server permissions then LiveKit local publish, never rely on old v1 media update endpoints.

API family: POST /api/v2/conferences/{id}/media-credential; participant current/role policies; LiveKit SDK

Required states and edge cases: Hardware disabled, device revoked, browser denial, host lock, remote mute, network reconnection.

Acceptance: Viewer cannot unmute/share via UI or patched client; enforced lock persists and is communicated.

F-12 Host media and participant controls

UI and behavior: Participant details, rename only if policy allows, role menu, mute, remove, ban, audio/video/share locks, stop active sharing, spotlight on/off. Clearly distinguish self mute from enforced lock and personal pin from shared spotlight.

API family: PATCH /api/v2/conferences/{id}/participants/{participantId}/role; POST /mute, /remove, /ban, /audio-lock, /video-lock, /screen-share-lock; DELETE matching locks; POST /screen-share/{participantId}/stop; POST /spotlights

Required states and edge cases: 403, 404, role-change race, cross-room moderation, provider 503, stale version.

Acceptance: An authorized host change updates business state, participant UI and provider permissions; removed participant loses media.

F-13 Text chat and private/breakout messages

UI and behavior: Main-room messages, private recipient picker only where allowed, breakout chat, history cursor/infinite scroll, sent/pending/failed states, moderator delete/restrict and announcements. Sender identity is always server derived.

API family: GET/POST /api/v2/conferences/{id}/messages; DELETE /messages/{messageId}; PATCH /chat-policy; POST/DELETE /participants/{participantId}/chat-restriction

Required states and edge cases: Rate-limit, chat disabled, private disabled, breakout isolation, cursor replay, duplicate submissions.

Acceptance: No use of v1 chat messages in a v2 room; no cross-breakout leakage; explicit migration rules for future Part V.

F-14 Raised hands and reactions

UI and behavior: Hand raise with persistent queue order, lower own/host lower another, emoji reactions transient overlay, assistive announcements, disabled policy states.

API family: GET/POST/DELETE /api/v2/conferences/{id}/hand-raise; DELETE /participants/{participantId}/hand-raise; POST /reactions

Required states and edge cases: Rapid toggles, rate limits, changed policy, reconnect.

Acceptance: Raised state survives transient reconnect; ephemeral reaction does not become a stored chat message.

F-15 Breakout room orchestration

UI and behavior: Host creates named/numbered rooms, MANUAL/RANDOM/BALANCED assignment, opens rooms, announces, moves participants, visits and leaves, countdown to close and safe return to main. Participants see transition and isolated media/chat.

API family: GET/POST /api/v2/conferences/{id}/breakouts; POST /assign, /move, /close-all, /announcement; room-level /open, /close, /enter, /announcement; PATCH /assignments; POST /leave

Required states and edge cases: Move race, provider failure, forced room closure, missing assignment, host visiting not assigned.

Acceptance: A participant is never left connected to two incompatible breakout media contexts.

F-16 Polls and quizzes

UI and behavior: Organizer builder for questions/options/visibility and quiz correct-answer metadata, draft/open/close/archive states; attendee ballot, anonymous handling, result-visibility rules.

API family: GET/POST /api/v2/conferences/{id}/polls; GET /polls/{pollId}; POST /open,/close,/archive,/responses; GET /results

Required states and edge cases: Already voted, closed poll, 429, hidden results, anonymous results, multi-question score.

Acceptance: Only one ballot per logical participant; display results according to server visibility, not client role guess.

F-17 Q&A

UI and behavior: Ask question with optional anonymity, vote/unvote, view sorted questions, pin/unpin, answer/dismiss/archive for moderators, unanswered/pinned tabs. Distinct from freeform chat.

API family: GET/POST /api/v2/conferences/{id}/questions; POST /questions/{questionId}/answer,/dismiss,/archive,/pin,/vote; DELETE /pin,/vote

Required states and edge cases: Rate limited, already answered, anonymous identity, moderation denied.

Acceptance: Anonymous identity not leaked; first committed answer authoritative.

F-18 Recording, consent and recording indicator

UI and behavior: Separate legal notice and affirmative consent per required type; host starts/stops only if allowed; show REQUESTED/RECORDING/STOPPING/PROCESSING/READY/FAILED; broadcast visible live indicator to all; no optimistic false recording state.

API family: GET/POST /api/v2/conferences/{id}/recordings; GET /recordings/{recordingId}; POST /stop; GET /consents/requirements; POST /consents; POST /consents/{consentType}/withdraw

Required states and edge cases: Not enabled, consent missing, provider 503, stale recording, retention expiry, concurrent starts.

Acceptance: Recording badge derives from authoritative recording/provider evidence, not button click.

F-19 Post-meeting outputs, reports and downloads

UI and behavior: End screen and past meeting detail: attendance, recordings, transcripts, authorized artifacts; generate reports, request expiring signed object URL just before download, do not persist signed URLs.

API family: GET /api/v2/conferences/{id}/attendance,/artifacts,/recordings,/transcripts; POST /artifacts/reports,/artifacts/{artifactId}/access; DELETE /artifacts/{artifactId}

Required states and edge cases: Report queued, processing, unauthorized artifact, expired URL, deletion and retention.

Acceptance: No direct storage bucket paths; artifact access is re-authorized each time.

F-20 Transcription, captions and intelligence (conditional)

UI and behavior: When enabled, caption language/speaker/time, live status and historical transcript, user toggle for captions, translation selection, explicit AI processing consent and disabled states. No promise that provider exists by default.

API family: POST /api/v2/conferences/{id}/transcription,/transcription/{jobId}/stop; GET /transcripts,/transcripts/{artifactId}; POST /translations; POST /intelligence/requests

Required states and edge cases: Provider unavailable, partial/live/final segments, private speech, consent revoked, translation queued.

Acceptance: Never show feature as live if flag/provider is absent; separate transcription from AI permissions.

F-21 Webinar and town hall (conditional)

UI and behavior: Attendee registration (approved fields only), confirmation/revocation, view-only audience, presenter promotion, host stage controls/backstage, scalable viewer layout with Q&A/polls.

API family: GET/POST /api/v2/meeting-spaces/{id}/registrations; GET/DELETE /registrations/{registrationId}; POST/DELETE /api/v2/conferences/{id}/backstage; PATCH participant role

Required states and edge cases: Capacity reached, registration disabled, event not started, view-only grant change, backstage RTC handoff.

Acceptance: Viewer cannot publish by default; promotion recomputes server RTC publish grants.

F-22 Healthcare / high-security profile (conditional)

UI and behavior: Appointment-context join and verified access, strong privacy language, restricted meeting title until authorized, stricter waiting room/consent/recording defaults; use generic profile resolver, not healthcare data model duplication.

API family: GET/PATCH /api/v2/meeting-spaces/{id}/profile-policy; GET /conferences/{id}/pre-join; POST /conferences/{id}/join and /consents

Required states and edge cases: External appointment authorization unavailable, restricted role, recording default disabled, identity mismatch.

Acceptance: No PHI in URLs/analytics/console; deny access safely, preserve minimal audit correlation.

F-23 Quality, connectivity and recovery

UI and behavior: Unobtrusive network health indicator and device troubleshooting; RTC stats samples for own session, authorized quality report and recovery after provider/WS failure. Never mistake media connectivity for conference admission.

API family: POST /api/v2/conferences/{id}/quality/samples; GET /quality; GET /events and canonical state; LiveKit RTC stats

Required states and edge cases: Offline, temporary provider errors, bad samples, 413/429, stale cursor, reconnect exhaustion.

Acceptance: No secret data in metrics; show retry and recovered state without duplicate meetings/sessions.

F-24 End meeting, leave and post-meeting transition

UI and behavior: Separate Leave (self disconnect; conference continues) and End for all (host operation). End must transition to ENDING, show progress, reject fresh joins, close RTC room, then reach ENDED.

API family: POST /api/v2/conferences/{id}/leave; POST /conferences/{id}/end; GET /conferences/{id}

Required states and edge cases: END pending/provider fault, tab closed, disconnect race, offline leave, host changed.

Acceptance: Leave never ends conference; end for all never claims completion until canonical end state.

6. Complete user journeys and state transitions

J-01 Guest via share link

6. Read identifier from link, call resolve, preserve allowed summary and selected conferenceId.
7. Render pre-join contract, collect guest displayName only for presentation and optional passcode/invitation/consent data.
8. POST join. If WAITING, store participantToken and enter waiting state without RTC media credential.
9. Subscribe SELF_ONLY scoped stream and fallback GET participants/me. On ADMITTED, obtain RTC permission/credential if needed and connect.
10. On leave call POST /leave; close media and release local session tokens according to retention policy.

J-02 Registered scheduled meeting

11. Authenticate account, list spaces, choose existing/new space.
12. Create conference occurrence, preserve conferenceId and schedule in space timezone.
13. Owner prepares/starts when allowed, shares link/code.
14. Participants join/are admitted according to policies; events update the roster.
15. Host ends; all attendees receive canonical ended and move to summary.

J-03 Instant meeting

16. POST /api/v2/instant-meetings returns space + LIVE conference.
17. Immediately join with POST /api/v2/conferences/{id}/join and preserve all join fields.
18. Navigate directly to /conferences/{conferenceId}/live; never rely on unconsumed legacy session storage.
19. Copy stable meetingCode link for invitees; use realtime and LiveKit.

J-04 Waiting room denial or ban

20. Show waiting with non-sensitive meeting information and a cancel/leave action.
21. On admission event refetch own participant state; on denied/banned navigate to distinct safe terminal screen.
22. Expired/revoked invite cannot be retried silently; allow user to return or obtain a new invitation.

J-05 Audio/video permission change

23. LiveKit local track toggles reflect latest JoinPermissions and server role/policy.
24. Host changes role/lock; event arrives to target and everyone permitted to see it.
25. Target immediately updates controls and grants; stop publishing forbidden tracks and show neutral policy explanation.

J-06 Reconnect with lost events

26. Capture last contiguous sequence per conference. Reconnect with new one-shot ticket and previous cursor when supported.
27. Replay GET /events?after=lastCommittedCursor before accepting later events, dedupe by eventId and detect gaps.
28. If replay unavailable/too old, re-fetch GET conference, participants/me, participants and collaboration; restore cursor safely.

J-07 Breakout transition

29. Host creates and assigns breakouts; participants see room prompt/state.
30. Backend authorizes move and updates provider routing; frontend leaves previous context and joins new media context.
31. Breakout-specific chat/permissions visible only in current context.
32. On close/countdown, return to main conference and restore stage layout.

J-08 Recording with consent

33. Pre-join informs each attendee of processing policies and gathers required consent.
34. Host requests start, UI shows REQUESTED then RECORDING only when confirmed.
35. All participants receive recording indicators and policy-change prompts.
36. Stop request transitions to STOPPING/PROCESSING; post-meeting view later shows READY / FAILED.

J-09 Webinar registration to presentation

37. Attendee completes approved registration; receives suitable admission link/token.
38. Attendee joins as VIEWER (subscribe-only) and interacts using allowed Q&A/polls/reactions.
39. Authorized host promotes attendee; role/permissions event updates client and RTC grants; presenter can publish only after approval.

J-10 Regulated appointment

40. External appointment authorization enters as opaque context, never as trust asserted by browser.
41. Server validates actor/context and enforces privacy-first summary and waiting policies.
42. Recording/transcription disabled by default unless explicitly permitted and consented; no clinical data leakage.

7. API integration architecture and type contracts

7.1 Client architecture and boundaries

Recommended stack: existing Next.js/React TypeScript application; route-level data loaders/React Query for canonical REST resources; typed operation clients generated from pinned meeting OpenAPI; LiveKit SDK solely for authorized RTC media; a dedicated conference realtime adapter for business state. This is a frontend design recommendation, not a backend requirement.

Module	Owned concerns / safeguards
api/authClient	v1 account auth, refresh coordination, logout, bearer and guest token separation
api/meetingV2Client	typed resource operations, safe error decoding, no arbitrary URLs, idempotent mutation keys where supported
domain/identities	MeetingSpaceId/ConferenceId/ParticipantId/ParticipantSessionId nominal types; no string interchange
domain/meetingPolicy	effective permissions, roles, meeting type, feature capabilities, visible actions
realtime/conferenceStream	ticket issuance, scoped websocket, cursor replay, dedupe, REST resync
rtc/livekitAdapter	connect/disconnect, devices/tracks, permissions, reissued credentials and RTC events
features/*	screen-specific hooks and UI; no backend security decisions in UI code
observability	correlation IDs, join latency, recovered events, redacted diagnostics

7.2 Canonical domain identifiers

Use nominal/brand-style TypeScript types (or equivalent) to prevent accidental swapping. For example: MeetingSpaceId, ConferenceId, ParticipantId and ParticipantSessionId. The app route for a live meeting is keyed by ConferenceId. The public share code is a resolver input only. Old/legacy IDs must be resolved, never guessed.

7.3 Verified key API request/response shapes

Operation	Key properties in current contract
POST /api/v2/meeting-spaces	CreateMeetingSpaceRequest: title required; optional timezone, meetingType, access/waiting policies, securityProfile, capacities, recording/transcription policies, org and role/collaboration policy.
PATCH /api/v2/meeting-spaces/{id}	UpdateMeetingSpaceRequest requires version; changed policy fields and optional passcode/clearPasscode; handle 409.
POST /api/v2/conferences/{id}/join	JoinConferenceRequest allows displayName, invitationToken, passcode, externalContext, consents, previousSessionId; unknown identity/role fields forbidden.
JoinConferenceResponse	conference, participant, rtcStatus, permissions, policy, mediaContext, realtime required; optional participantSession, participantToken and rtc credential.
GET /conferences/{id}/pre-join	PreJoinResponse includes meetingSpaceId, conferenceId, meetingType, conferenceStatus, joinable, access, notices, consents, media, displayName and devices.
POST /conferences/{id}/realtime/ticket	Returns ticket, expiresAt, websocketPath, cursor, scope (FULL / SELF_ONLY); one-shot ticket bound to conference and short TTL.
POST /conferences/{id}/waiting-room/admit	AdmitRequest: all or participantIds; confirmation and idempotent UI needed.
POST /conferences/{id}/messages	SendMessageRequest requires body; optional announcement, breakoutId or recipientParticipantId; actor identity derived on server.
PUT /meeting-spaces/{id}/recurrence	SetRecurrenceRequest requires rrule, startLocal, durationMinutes; IANA meeting timezone remains authoritative.
POST /conferences/{id}/recordings	StartRecordingRequest optional idempotencyKey/layout; never selects provider/storage internals.
POST /conferences/{id}/consents	RecordConsentRequest requires consentVersion and decisions array; must use the caller's own identity.

7.4 Authentication and credential handling

43. Signed-in user calls v1 auth to obtain account access token. User profile is GET /api/v1/users/me. Refresh with POST /api/v1/auth/refresh; prevent simultaneous refresh storms.
44. Guest who receives a v2 participantToken sends it only as X-Participant-Token to conference-scoped operations; never substitute a guest token into global account identity.
45. RTC credential is a provider-specific short-lived credential used for LiveKit media, not the same as API access, participant token or WebSocket ticket.
46. Realtime ticket is one-shot, conference-bound and expires quickly; mint only immediately before WebSocket connection.
47. Never log tokens, store them in links, render them in analytics, send them to third-party scripts or expose them via console/debug UI. Prefer appropriate memory/session scope over long-lived persistence.

7.5 API errors and recovery contract

v2 standard error envelope: {success:false,error:{code,message,requestId,retryAfterSeconds?}}. Interpret by error.code, not free-form message text. Preserve requestId for safe support reporting. The backend PDF Appendix C lists suggested codes; newer OpenAPI is definitive for the current enum and per-operation responses.

Error / state	Required frontend behavior
MEETING_NOT_FOUND / CONFERENCE_NOT_FOUND	Show safe not-found state; no private metadata leak.
MEETING_PASSCODE_REQUIRED / INVALID	Prompt securely, label wrong passcode, allow retry within rate policy.
INVITATION_INVALID / EXPIRED	Invite expired/revoked screen; never auto-repeat with stale token.
MEETING_ACCESS_DENIED / PERMISSION_DENIED	Access denied or no permission; no retry loop.
CONFERENCE_LOCKED / NOT_LIVE / STATE_CONFLICT	Show status-dependent message, refresh canonical conference.
PARTICIPANT_BANNED	End current media and show no rejoin option with stale credential.
RTC_PROVIDER_UNAVAILABLE	Preserve authorized state; offer retry/backoff without claiming media connected.
RATE_LIMITED / retryAfterSeconds	Cooldown timer and disabled send/retry; no aggressive auto-retry.
Version / conflict HTTP 409	Fetch latest entity, surface changed data, let user reapply intent.
Feature disabled or provider not configured	Hide or disable associated feature with a clear supported alternative.

8. Realtime, RTC and reliability design

8.1 Separate canonical meeting events from LiveKit tracks

The server controls conference lifecycle, admission, role, membership, bans, locks, recording and policy decisions. LiveKit enforces media and emits local track/connection events. Neither LiveKit presence nor a successful HTTP 200 alone proves admission. The UI must reconcile both planes.

8.2 Conference event protocol

48. After authorize/join or waiting response, obtain short-lived ticket from POST /realtime/ticket (when realtime.enabled).
49. Connect GET /realtime via websocketPath with ticket and previous cursor; scope FULL or SELF_ONLY.
50. Process a stable envelope with eventId, eventType, schemaVersion, occurredAt, payload and sequence; track event IDs and monotonic conference sequence.
51. On sequence gap or dropped connection use /events?after=<cursor>; then canonical conference, current participant, roster and collaboration GETs if replay is incomplete.
52. Use exponential bounded reconnect and visible “Reconnecting…” UI. Never clear admission/token state automatically on a transient RTC or WS disconnection.
53. On conference.ended, participant.removed, participant.banned, denied or provider terminal state, stop media and transition safely.
54. On page close, attempt explicit /leave where practical, while accepting that browser termination is not a reliable guaranteed request.

8.3 Critical realtime event families from backend specification

Event	Immediate frontend effect
conference.started / locked / unlocked / ended	Update status, toolbar, join gating, meeting-ended transition.
participant.requested_join / admitted / denied	Update waiting queue; waiting participant transitions on self state.
participant.joined / left	Refresh roster, session count, media tile mapping.
participant.role_changed / media_permission_changed	Refresh permissions and reconfigure publishing UI/tracks.
participant.removed / banned	Disconnect affected participant and clear issued media credentials.
screenshare.started / ended	Switch stage layout / shared screen spotlight.
recording.started / stopped / ready / failed	Synchronize visible indicator, controls and artifacts.
breakout.opened / assignment_changed / closed	Route participants to allowed media/chat context.
poll.started / closed	Update ballot visibility, status and results eligibility.

8.4 Browser media UX requirements

- Preview/test camera and mic before joining; show browser permission prompts and device fallback instructions.
- Surface hardware-change, device selection, no camera/microphone, speaker-output support and permission revocation.
- Separate local mute, server-mute, media permission lock and role/view-only denial with distinct iconography and accessible tooltips.
- Screen-share stage layout should prioritize shared content but allow pin/fullscreen and mobile reflow without overflow.
- When browser/screen share stops spontaneously, synchronize provider state and update attendee view, but avoid false success of v1 state-update APIs.

9. Security, privacy, accessibility and responsive behavior

9.1 Frontend security obligations

- DisplayName, email, meetingCode and invitation links never grant identity/host rights; all actions rely on server-resolved ActorContext.
- Do not expose internal/admin migration APIs in a user-facing screen or ship their credentials in browser code.
- Do not persist plain passcodes or raw invite tokens after their intended use; token returned once for invite creation.
- Prohibit unsafe HTML in chat, Q&A, transcript and descriptions; use semantic text rendering and safe links.
- Use least-privilege telemetry: only conferenceId and participantSessionId as correlation keys; redact contact and clinical data.
- Handle healthcare/private meeting title as restricted until authorized, even in browser document title, recents and analytics.
- Do not cache signed artifact URLs beyond their brief lifespan; request just-in-time and clear when user logs out.
- No UI authorization shortcut may reactivate a removed/denied/banned or ended media connection.

9.2 Accessibility acceptance

- Aim for WCAG 2.2 AA on the web frontend; keyboard access for every dialog, menu, tile, panel and breakout transition.
- All mic/camera/share/hand/recording states have visible text or screen-reader labels beyond color alone.
- Focus management on join flow, waiting-room result, modal close, toolbar and breakout movement.
- Live-region announcements are polite and rate-limited; participant churn/reactions must not overwhelm assistive tech.
- Captions, Q&A, polls, webinar viewer mode and consent controls must function at 200% zoom and on mobile.
- Dialogs support ESC where safe, trap focus appropriately, and explicitly confirm destructive host actions.

9.3 Responsive / performance requirements

- Support desktop, tablet and narrow-phone widths with adaptive grid/stage, condensed host actions and non-overlapping side panels.
- Keep media controls visible while secondary content scrolls; provide keyboard and touch alternatives to hover-only controls.
- Use virtualization for large rosters, chat history and attendee registration lists; cursor paginate through documented APIs.
- Avoid refetching every resource on every realtime event; invalidate relevant query families and coalesce bursts.
- Record performance measures: time-to-prejoin, click-to-first-media, reconnection time, meeting action confirmation delay, and WebSocket replay completeness.

10. Feature-gated modes and conditional experiences

Feature family	What frontend does if enabled	What to do if disabled / unavailable
Recording	Show consent, controls, indicator, post-meeting READY/download	No fake Record control; display policy-based unavailability.
Transcription/captions	Caption control, transcript views, language/translation selectors	Do not claim live captions if provider is disabled.
Intelligence/AI	Consent and explicit output request after source artifact READY	Defer UI, do not generate local summaries as backend outputs.
Webinar/town hall	Registration, view-only stage, backstage, Q&A and polls	Fall back to STANDARD meeting behavior only when meeting policy allows.
Healthcare/high security	Appointment validation, restricted metadata, default waiting and privacy controls	Reject/disable unauthorized access; never show public meeting defaults.
Quality telemetry	Self-session samples, organizer health summary	Local network diagnostics only; do not claim server monitoring.
External calendar sync	Optional authorized provider linking and sync status	Internal recurrence and schedule must still work.
Unified Messaging	Integrate only via signed-off separate messaging frontend contract and migration flag	Retain the published v2 conference chat integration pending explicit migration.

11. Acceptance criteria and end-to-end certification

11.1 Required front-end E2E scenarios

55. AT-01 Create instant meeting, join with valid RTC grant, copy link, invite guest, guest enters and leaves.
56. AT-02 Create scheduled space and separate occurrence; create recurring timezone-aware series with DST edge cases.
57. AT-03 Cancel one recurring occurrence and verify series / past data remain unchanged.
58. AT-04 Legacy meeting URL resolves to new canonical space and conference without changing actor identity.
59. AT-05 Public-passcode meeting: required / invalid / correct password plus rate-limited attempts.
60. AT-06 Invite-only meeting: valid, revoked, expired and exhausted invitation experiences.
61. AT-07 Organization/domain/appointment restrictions reject unauthorized actor before RTC credential issuance.
62. AT-08 Guest displays another user's name but does not gain their session/role.
63. AT-09 Guest enters WAITING without provider media grant, host admits, guest connects once.
64. AT-10 Host denies with safe message; denied/removed/banned user cannot use stale credential.
65. AT-11 Host admits selected/all participants and queue updates without duplicates after retries.
66. AT-12 Host locks/unlocks LIVE room and concurrent join obeys server policy.
67. AT-13 Co-host receives delegated capabilities; viewer remains subscribe-only until promoted.
68. AT-14 Role change and provider permissions synchronize across two independent browser clients.
69. AT-15 Host mutes/locks participant mic/video/share and participant UI reflects enforced state.
70. AT-16 Browser media device unplug/permissions denied and connection recovery show correct UX.
71. AT-17 Host stop active screen share; person cannot bypass screen-share-lock without grant.

72. AT-18 Main room chat/history cursor, private chat and breakout chat adhere to scope and moderation.
73. AT-19 Chat disabled/restricted, rate-limited, duplicate submission and message removal handled.
74. AT-20 Hand raise ordering persists over brief reconnect; host lowers others; reactions remain ephemeral.
75. AT-21 Breakout create/assign/open/move/announce/close with server-authorized media isolation.
76. AT-22 Poll draft/open/respond/close/archive including anonymous mode and delayed result visibility.
77. AT-23 Q&A create/vote/answer/pin/dismiss/archive with anonymous identity protection.
78. AT-24 Recording consent before join and mid-call change, state indicator and READY/FAILED artifact.
79. AT-25 Recording provider unavailable does not claim recording active or interrupt conference.
80. AT-26 Artifact access requested with short-lived signed URL; expired/deleted/unauthorized paths safe.
81. AT-27 Transcription/captions conditional path and disabled-provider states render correctly.
82. AT-28 Webinar registration, view-only join, backstage, promoted presenter gets valid grant.
83. AT-29 Healthcare appointment context fail/deny; restricted meeting title not leaked to UI/logs.
84. AT-30 WS drop, duplicate/out-of-order events and cursor replay recover authoritative state.
85. AT-31 Conference ends during join; no new media grant; all connected clients exit safely.
86. AT-32 Host leave does not end meeting; host end terminates media for everyone.
87. AT-33 Anonymous guest token not lost after refresh/reconnect; cross-conference token fails.
88. AT-34 409 stale version update refetches and prompts user to reapply changes.
89. AT-35 Responsive and WCAG keyboard/screen-reader audit covers lobby, live room and dialogs.
90. AT-36 Browser refresh mid-conference rehydrates only legitimate identity/session and keeps no stale unauthorized grant.
91. AT-37 Local feature flag OFF preserves safe UI and never leaks inaccessible endpoints.
92. AT-38 Production-like smoke tests use exact deployed URL, enabled flags and live provider state; no assumption from Swagger listing alone.

11.2 Shipping gates

Gate	Required evidence
Contract parity	Frontend generated API client matches pinned Swagger version; no undocumented request properties / path assumptions.
State authority	No v2 ConferenceId / MeetingSpaceId mix-ups; waiting participant has no RTC join.
Permissions	Role/action matrix tested; viewer/guest restrictions and backend 403/404 response flows covered.
Multi-browser RTC	Host + attendee + guest + viewer; active media, share, role change, waiting and end.
Realtime recovery	Duplicate/gapped event playback, cursor expiry fallback, late admission and disconnected client recovery.
Security / privacy	IDOR negative cases, logs/token audits, XSS rendering, signed URLs, restricted metadata.
Accessibility and responsive	Automated + manual keyboard, screen reader, touch and device matrix.
Operations	Join/reconnect timing, correlation requests and relevant error dashboards.
Rollout/rollback	Canary and safe feature-disable without reverting to insecure v1 admission.

12. Delivery plan, dependencies and decisions

Wave	Frontend deliverable	Exit gate
W0 Foundation	Pinned contract/type generation, identity model, auth, error handling, route skeleton	Type/client parity, no ID mix-up
W1 Meeting basics	Dashboard, create/edit, instant, schedule, links, profile/space details	Owner/guest E2E and legacy link resolver
W2 Admission & RTC	Pre-join, waiting, permissions, real media, leave/end	Full registered/guest/viewer multi-browser RTC E2E
W3 Host controls	Roster, moderation, locks, roles, provider/state reconciliation	Denied/ban/role/media scenarios
W4 Collaboration	Chat, hands/reactions, polls, Q&A, breakout manager	Room privacy/ordering and E2E
W5 Conditional modes	Recording/artifacts/consent, webinar, profiles, captions if enabled	Per-capability environment/provider certification
W6 Hardening	Accessibility, responsive, telemetry, failure/recovery, performance and canary	Release evidence, sign-off, rollback readiness

12.1 Decisions to resolve before development

- D-01: Confirm which public origin is the browser API authority for /api/v2 meeting operations in staging and production; the six-service Swagger selector is NOT a reverse proxy.

- D-02: Confirm pinned GitHub SHA of the generated v1.2.0 OpenAPI and whether downstream API Gateway public contract (130 v2 operations) or Meeting Service public contract (193) is the frontend integration target.
- D-03: Confirm enabled runtime flags/provider capacity for recording, webinar, healthcare, quality, transcription and intelligence, per environment.
- D-04: Confirm guest token and participant session persistence policy across refresh, tab close and multiple devices.
- D-05: Confirm Phase V meeting-chat migration cutover. Use v2 conference messages until contract + deployment and parity are approved.
- D-06: Confirm any licensed design assets, accessibility color tokens and the target browser/mobile device support matrix.
- D-07: Confirm whether organization admin/member UI and v1 developer API keys/webhook panels belong to this meeting frontend or separate consoles.
- D-08: Confirm invitation delivery/notification/email expectations; URL token exposure strategy and share behavior.
- D-09: Confirm viewer/interactive capacity ceilings and breakout/recording policies for each meeting type.
- D-10: Confirm backend contract for checking enabled capabilities (if no discovery endpoint exists, use server-governed environment config and safe feature-disabled handling).

Appendix A — Complete Meeting Service operation reference

The following 205-row inventory was transcribed from the checked-in OpenAPI v1.2.0 source contract reviewed 25 Oct 2026. Descriptions in the “What it does” column are concise editorial paraphrases of the operation summaries, not an exported exhaustive schema. “Where used” and “Frontend contract” are frontend requirements/design guidance, not assertions that the backend currently implements those UI screens. HTTP response code sets, request DTO names and security alternatives were extracted from the versioned OpenAPI; consult the canonical contract for parameter/property constraints, bodies, examples, and error-code mappings. "not-declared" is not itself a proof that runtime access is anonymous.

Documentation audience: 205 source operations; filtering out admin paths and /health yields 193 meeting operations in the public Meeting Service Swagger as reported in production on 25 Oct 2026. A separate API Gateway meetings bundle exposes 130 v2 operations. These are different documentation/application surfaces.

A.1 Meeting spaces & discovery (11 operations)

Consumer classification: Implement. Primary UI: Dashboard, meeting creation wizard, details, invitation links and old-link routing.

`POST /api/v2/instant-meetings`

What it does: Create meeting space and live conference atomically. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: InstantMeetingRequest **Swagger security:** bearer_auth OR platformCredential(meetings.write) **HTTP:** 201,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/meeting-spaces`

What it does: List caller-owned or member meeting spaces. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: no-json-body **Swagger security:** bearer_auth OR platformCredential(meetings.read) **HTTP:** 200,401 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/meeting-spaces`

What it does: Create owned persistent meeting space. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: CreateMeetingSpaceRequest **Swagger security:** bearer_auth OR platformCredential(meetings.write) **HTTP:** 201,400,401,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/meeting-spaces/{id}`

What it does: Read meeting space by id or legacy or current code. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: no-json-body **Swagger security:** anonymous OR bearer_auth OR platformCredential(meetings.read) **HTTP:** 200,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PATCH /api/v2/meeting-spaces/{id}`

What it does: Update meeting space with optimistic version. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: UpdateMeetingSpaceRequest **Swagger security:** bearer_auth OR platformCredential(meetings.write) **HTTP:** 200,403,409 **Failure/state:** Handle stale version/conflict; refetch canonical state before retry.

`POST /api/v2/meeting-spaces/{id}/archive`

What it does: Soft archive meeting space with history preserved. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: VersionedRequest **Swagger security:** bearer_auth OR platformCredential(meetings.write) **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/meeting-spaces/{id}/conferences`

What it does: List scheduled live and past occurrences. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: no-json-body **Swagger security:** bearer_auth OR platformCredential(meetings.read) **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/meeting-spaces/{id}/conferences`

What it does: Create scheduled conference occurrence. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: CreateConferenceRequest **Swagger security:** bearer_auth OR platformCredential(meetings.write) **HTTP:** 201,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/meeting-spaces/{id}/duplicate`

What it does: Duplicate space configuration as new owned space. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: DuplicateMeetingSpaceRequest **Swagger security:** bearer_auth **HTTP:** 201,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/meeting-spaces/{id}/owner-transfer`

What it does: Transfer meeting space ownership. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: OwnerTransferRequest **Swagger security:** bearer_auth **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/resolve/{identifier}`

What it does: Resolve legacy or current link to space and conference. **Frontend:** Dashboard, meeting creation wizard, details, invitation links and old-link routing.

Request: no-json-body **Swagger security:** anonymous OR bearer_auth **HTTP:** 200,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

A.2 Conference lifecycle (10 operations)

Consumer classification: Implement. Primary UI: Meeting occurrence details, host start/end/lock flows and participant leave.

`GET /api/v2/conferences/{id}`

What it does: Read canonical conference status and authorized summary. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: no-json-body **Swagger security:** anonymous OR bearer_auth OR participant_token OR platformCredential(meetings.read) **HTTP:** 200,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/conferences/{id}/cancel`

What it does: Cancel unstarted conference occurrence. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.write) **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/end`

What it does: End conference and initiate RTC termination. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.write) **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/join`

What it does: Join or reconnect subject to identity access admission and provider policy. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: JoinConferenceRequest **Swagger security:** anonymous OR bearer_auth OR participant_token **HTTP:** 200,401,403,409,410,503 **Failure/state:** Map access/passcode/locked/banned/capacity/provider errors to distinct screens; never treat WAITING as joined.

`POST /api/v2/conferences/{id}/leave`

What it does: Close own active participant session or sessions. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: LeaveConferenceRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/lock`

What it does: Lock live conference against new nonexempt entries. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/prepare`

What it does: Prepare RTC room and transition to waiting state. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/reschedule`

What it does: Reschedule one unstarted conference occurrence. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: RescheduleConferenceRequest **Swagger security:** bearer_auth **HTTP:** 200,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/start`

What it does: Start authorized prepared conference. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.write) **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/unlock`

What it does: Unlock live conference. **Frontend:** Meeting occurrence details, host start/end/lock flows and participant leave.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.3 Scheduling & recurrence (5 operations)

Consumer classification: Implement. Primary UI: Schedule wizard, recurrence builder and occurrence exceptions.

`DELETE /api/v2/meeting-spaces/{id}/recurrence`

What it does: End recurrence while preserving prior occurrences. **Frontend:** Schedule wizard, recurrence builder and occurrence exceptions.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 204,403 **Failure/state:** Handle stale version/conflict; refetch canonical state before retry.

`GET /api/v2/meeting-spaces/{id}/recurrence`

What it does: Read active recurring schedule rule. **Frontend:** Schedule wizard, recurrence builder and occurrence exceptions.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PUT /api/v2/meeting-spaces/{id}/recurrence`

What it does: Create or replace RFC5545 recurring rule. **Frontend:** Schedule wizard, recurrence builder and occurrence exceptions.

Request: SetRecurrenceRequest **Swagger security:** bearer_auth **HTTP:** 200,400,403 **Failure/state:** Handle stale version/conflict; refetch canonical state before retry.

`GET /api/v2/meeting-spaces/{id}/recurrence/exceptions`

What it does: List one-off schedule exceptions. **Frontend:** Schedule wizard, recurrence builder and occurrence exceptions.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PUT /api/v2/meeting-spaces/{id}/recurrence/exceptions`

What it does: Cancel or reschedule one series occurrence. **Frontend:** Schedule wizard, recurrence builder and occurrence exceptions.

Request: RecurrenceExceptionRequest **Swagger security:** bearer_auth **HTTP:** 200,400,403,409 **Failure/state:** Handle stale version/conflict; refetch canonical state before retry.

A.4 Invitations (4 operations)

Consumer classification: Implement. Primary UI: Invite dialog, expiring link panel, redemption and revoke confirmation.

`POST /api/v2/invitations/{token}/resolve`

What it does: Resolve invitation token read-only. **Frontend:** Invite dialog, expiring link panel, redemption and revoke confirmation.

Request: no-json-body **Swagger security:** anonymous **HTTP:** 200,410 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/meeting-spaces/{id}/invitations`

What it does: List invitation metadata without tokens. **Frontend:** Invite dialog, expiring link panel, redemption and revoke confirmation.

Request: no-json-body **Swagger security:** bearer_auth OR platformCredential(meetings.read) **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/meeting-spaces/{id}/invitations`

What it does: Issue scoped expiring invitation token once. **Frontend:** Invite dialog, expiring link panel, redemption and revoke confirmation.

Request: CreateInvitationRequest **Swagger security:** bearer_auth OR platformCredential(meetings.write) **HTTP:** 201,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/meeting-spaces/{id}/invitations/{invitationId}/revoke`

What it does: Revoke an invitation. **Frontend:** Invite dialog, expiring link panel, redemption and revoke confirmation.

Request: no-json-body **Swagger security:** bearer_auth OR platformCredential(meetings.write) **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.5 Meeting membership (4 operations)

Consumer classification: Implement. Primary UI: Persistent member roster and role administration.

`GET /api/v2/meeting-spaces/{id}/members`

What it does: List authorized persistent members. **Frontend:** Persistent member roster and role administration.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/meeting-spaces/{id}/members`

What it does: Create authorized member role grant. **Frontend:** Persistent member roster and role administration.

Request: AddMemberRequest **Swagger security:** bearer_auth **HTTP:** 201,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/meeting-spaces/{id}/members/{memberId}`

What it does: Revoke meeting space membership. **Frontend:** Persistent member roster and role administration.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Handle stale version/conflict; refetch canonical state before retry.

`PATCH /api/v2/meeting-spaces/{id}/members/{memberId}`

What it does: Update authorized persistent member role. **Frontend:** Persistent member roster and role administration.

Request: UpdateMemberRequest **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Handle stale version/conflict; refetch canonical state before retry.

A.6 Organizations & tenancy (3 operations)

Consumer classification: Conditional/organization UI. Primary UI: Organization selector, member administration for authorized managers.

`GET /api/v2/me/organizations`

What it does: List caller organizations. **Frontend:** Organization selector, member administration for authorized managers.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`GET /api/v2/organizations/{id}/members`

What it does: List organization members with admin authorization. **Frontend:** Organization selector, member administration for authorized managers.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PUT /api/v2/organizations/{id}/members/{userId}`

What it does: Add or change organization membership. **Frontend:** Organization selector, member administration for authorized managers.

Request: UpsertOrganizationMemberRequest **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.7 Collaboration policy & pre-join (3 operations)

Consumer classification: Implement. Primary UI: Pre-join lobby, device selector and live collaboration policies.

`GET /api/v2/conferences/{id}/collaboration`

What it does: Get canonical collaboration state. **Frontend:** Pre-join lobby, device selector and live collaboration policies.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PATCH /api/v2/conferences/{id}/collaboration-policy`

What it does: Update versioned live collaboration policy. **Frontend:** Pre-join lobby, device selector and live collaboration policies.

Request: CollaborationPolicyUpdateRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,400,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/conferences/{id}/pre-join`

What it does: Get complete pre-join policy and consent contract. **Frontend:** Pre-join lobby, device selector and live collaboration policies.

Request: no-json-body **Swagger security:** anonymous OR bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

A.8 Admission & waiting room (4 operations)

Consumer classification: Implement. Primary UI: Attendee waiting state; organizer queue and bulk decisions.

`POST /api/v2/conferences/{id}/participants/{participantId}/return-to-waiting`

What it does: Return admitted participant to waiting state. **Frontend:** Attendee waiting state; organizer queue and bulk decisions.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/conferences/{id}/waiting-room`

What it does: List current waiting participants. **Frontend:** Attendee waiting state; organizer queue and bulk decisions.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.read) **HTTP:** 200,403 **Failure/state:** Handle 403/404/409; refresh queue and self admission state after changes.

`POST /api/v2/conferences/{id}/waiting-room/admit`

What it does: Admit one or many eligible waiting participants. **Frontend:** Attendee waiting state; organizer queue and bulk decisions.

Request: AdmitRequest **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.write) **HTTP:** 200,403,404,409 **Failure/state:** Handle 403/404/409; refresh queue and self admission state after changes.

`POST /api/v2/conferences/{id}/waiting-room/deny`

What it does: Deny one or many waiting participants. **Frontend:** Attendee waiting state; organizer queue and bulk decisions.

Request: DenyRequest **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.write) **HTTP:** 200,404 **Failure/state:** Handle 403/404/409; refresh queue and self admission state after changes.

A.9 Participants & moderation (5 operations)

Consumer classification: Implement. Primary UI: Participant roster, role picker, moderation actions and attendee state.

`GET /api/v2/conferences/{id}/participants`

What it does: List admitted conference participants. **Frontend:** Participant roster, role picker, moderation actions and attendee state.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.read) **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`GET /api/v2/conferences/{id}/participants/me`

What it does: Read own participant role and admission state. **Frontend:** Participant roster, role picker, moderation actions and attendee state.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/conferences/{id}/participants/{participantId}/ban`

What it does: Ban participant from conference or space. **Frontend:** Participant roster, role picker, moderation actions and attendee state.

Request: BanParticipantRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/participants/{participantId}/remove`

What it does: Remove participant and disconnect media. **Frontend:** Participant roster, role picker, moderation actions and attendee state.

Request: RemoveParticipantRequest **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.write) **HTTP:** 200,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`PATCH /api/v2/conferences/{id}/participants/{participantId}/role`

What it does: Grant authorized participant role change. **Frontend:** Participant roster, role picker, moderation actions and attendee state.

Request: RoleChangeRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.10 RTC media control (11 operations)

Consumer classification: Implement. Primary UI: Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

`POST /api/v2/conferences/{id}/media-credential`

What it does: Refresh media credential for caller session. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: MediaCredentialRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404,409,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/conferences/{id}/participants/{participantId}/audio-lock`

What it does: Lift server-enforced microphone lock. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/participants/{participantId}/audio-lock`

What it does: Apply server-enforced microphone lock. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/participants/{participantId}/mute`

What it does: Mute participant via provider control. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: MuteRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/conferences/{id}/participants/{participantId}/screen-share-lock`

What it does: Lift screen sharing lock. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/participants/{participantId}/screen-share-lock`

What it does: Apply server-enforced screen share lock. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/conferences/{id}/participants/{participantId}/video-lock`

What it does: Lift server-enforced camera lock. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/participants/{participantId}/video-lock`

What it does: Apply server-enforced camera lock. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/screen-share/{participantId}/stop`

What it does: Stop another participant live screen share. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/spotlights`

What it does: Set shared spotlight participant. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: SpotlightRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/conferences/{id}/spotlights/{participantId}`

What it does: Remove shared spotlight. **Frontend:** Mic/camera/screenshare toolbar, permission locks, spotlight, safe credential renewal.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.11 Realtime & recovery (3 operations)

Consumer classification: Implement. Primary UI: Invisible conference event client with reconnect and REST reconciliation.

`GET /api/v2/conferences/{id}/events`

What it does: Replay committed realtime events by cursor. **Frontend:** Invisible conference event client with reconnect and REST reconciliation.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Reconnect with cursor and REST snapshot; treat stale cursor as resync.

`GET /api/v2/conferences/{id}/realtime`

What it does: Subscribe to authenticated WebSocket state changes. **Frontend:** Invisible conference event client with reconnect and REST reconciliation.

Request: no-json-body **Swagger security:** anonymous OR bearer_auth OR participant_token **HTTP:** 101,400,401,403,404 **Failure/state:** Reconnect with cursor and REST snapshot; treat stale cursor as resync.

`POST /api/v2/conferences/{id}/realtime/ticket`

What it does: Issue single-use short-lived WebSocket ticket. **Frontend:** Invisible conference event client with reconnect and REST reconciliation.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,401,403,404 **Failure/state:** Reconnect with cursor and REST snapshot; treat stale cursor as resync.

A.12 Meeting chat (6 operations)

Consumer classification: Implement. Primary UI: Main/breakout/private conversation panels and moderated chat.

`PATCH /api/v2/conferences/{id}/chat-policy`

What it does: Change chat-specific collaboration policy. **Frontend:** Main/breakout/private conversation panels and moderated chat.

Request: ChatPolicyUpdateRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/conferences/{id}/messages`

What it does: Retrieve cursor-paginated conference messages. **Frontend:** Main/breakout/private conversation panels and moderated chat.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/conferences/{id}/messages`

What it does: Send authorized conference breakout or private message. **Frontend:** Main/breakout/private conversation panels and moderated chat.

Request: SendMessageRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 201,400,403,404,409,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/conferences/{id}/messages/{messageId}`

What it does: Delete or hide own/moderated message. **Frontend:** Main/breakout/private conversation panels and moderated chat.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/conferences/{id}/participants/{participantId}/chat-restriction`

What it does: Restore participant chat permission. **Frontend:** Main/breakout/private conversation panels and moderated chat.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/participants/{participantId}/chat-restriction`

What it does: Restrict participant from chat. **Frontend:** Main/breakout/private conversation panels and moderated chat.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.13 Reactions & raised hands (5 operations)

Consumer classification: Implement. Primary UI: Meeting toolbar, raised-hand queue and brief reaction overlays.

`DELETE /api/v2/conferences/{id}/hand-raise`

What it does: Lower own hand. **Frontend:** Meeting toolbar, raised-hand queue and brief reaction overlays.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,401,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/conferences/{id}/hand-raise`

What it does: List raised hands oldest first. **Frontend:** Meeting toolbar, raised-hand queue and brief reaction overlays.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/conferences/{id}/hand-raise`

What it does: Raise own hand. **Frontend:** Meeting toolbar, raised-hand queue and brief reaction overlays.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/conferences/{id}/participants/{participantId}/hand-raise`

What it does: Host lower participant hand. **Frontend:** Meeting toolbar, raised-hand queue and brief reaction overlays.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/reactions`

What it does: Send policy-permitted ephemeral reaction. **Frontend:** Meeting toolbar, raised-hand queue and brief reaction overlays.

Request: ReactionRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.14 Breakout rooms (12 operations)

Consumer classification: Feature-gated. Primary UI: Host room manager, participant breakout transport, countdown and announcements.

`GET /api/v2/conferences/{id}/breakouts`

What it does: List conference breakout rooms. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/conferences/{id}/breakouts`

What it does: Create breakout rooms. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: CreateBreakoutsRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 201,400,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/announcement`

What it does: Broadcast host announcement to open breakouts. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: BreakoutAnnouncementRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,409,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/assign`

What it does: Assign participants manually randomly or balanced. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: AssignBreakoutsRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/close-all`

What it does: Close all open breakouts. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: CloseBreakoutRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/leave`

What it does: Leave privileged breakout visit. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/move`

What it does: Move participant between main and breakout rooms. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: MoveParticipantRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/{breakoutId}/announcement`

What it does: Send announcement to one breakout room. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: BreakoutAnnouncementRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`PATCH /api/v2/conferences/{id}/breakouts/{breakoutId}/assignments`

What it does: Add or remove participants assigned to breakout. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: RoomAssignmentsPatch **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/{breakoutId}/close`

What it does: Close breakout with optional countdown. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: CloseBreakoutRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/{breakoutId}/enter`

What it does: Privileged breakout visit. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/breakouts/{breakoutId}/open`

What it does: Open breakout and route assigned participants. **Frontend:** Host room manager, participant breakout transport, countdown and announcements.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.15 Polls & quizzes (8 operations)

Consumer classification: Feature-gated. Primary UI: Poll editor, live participation panel, results and visibility rules.

`GET /api/v2/conferences/{id}/polls`

What it does: List visible polls and quizzes. **Frontend:** Poll editor, live participation panel, results and visibility rules.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/polls`

What it does: Create draft poll or quiz. **Frontend:** Poll editor, live participation panel, results and visibility rules.

Request: CreatePollRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 201,400,403 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`GET /api/v2/conferences/{id}/polls/{pollId}`

What it does: Read one visible poll. **Frontend:** Poll editor, live participation panel, results and visibility rules.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/polls/{pollId}/archive`

What it does: Archive completed poll or cancel draft. **Frontend:** Poll editor, live participation panel, results and visibility rules.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/polls/{pollId}/close`

What it does: Close open poll permanently. **Frontend:** Poll editor, live participation panel, results and visibility rules.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/polls/{pollId}/open`

What it does: Open draft poll for responses. **Frontend:** Poll editor, live participation panel, results and visibility rules.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/polls/{pollId}/responses`

What it does: Submit own one-ballot poll answer. **Frontend:** Poll editor, live participation panel, results and visibility rules.

Request: SubmitPollResponseRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 201,400,404,409,429 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`GET /api/v2/conferences/{id}/polls/{pollId}/results`

What it does: Read results subject to release policy. **Frontend:** Poll editor, live participation panel, results and visibility rules.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

A.16 Q&A (9 operations)

Consumer classification: Feature-gated. Primary UI: Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

`GET /api/v2/conferences/{id}/questions`

What it does: List Q&A questions with pins and vote ordering. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/questions`

What it does: Submit Q&A question. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: SubmitQuestionRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 201,403,429 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/questions/{questionId}/answer`

What it does: Answer Q&A question. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: AnswerQuestionRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/questions/{questionId}/archive`

What it does: Archive Q&A question. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/questions/{questionId}/dismiss`

What it does: Dismiss Q&A question. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`DELETE /api/v2/conferences/{id}/questions/{questionId}/pin`

What it does: Remove host question pin. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/questions/{questionId}/pin`

What it does: Pin Q&A question. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,409 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`DELETE /api/v2/conferences/{id}/questions/{questionId}/vote`

What it does: Remove own question upvote. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

`POST /api/v2/conferences/{id}/questions/{questionId}/vote`

What it does: Upvote question. **Frontend:** Q&A viewer, create, vote, answer, pin, dismiss and moderation tools.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404,409 **Failure/state:** Respect visibility, role gates, rate limits and state transitions.

A.17 Consent (3 operations)

Consumer classification: Implement. Primary UI: Pre-join processing notices, consent screens and in-room withdrawal UI.

`POST /api/v2/conferences/{id}/consents`

What it does: Record caller's own consent decisions. **Frontend:** Pre-join processing notices, consent screens and in-room withdrawal UI.

Request: RecordConsentRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,400,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/conferences/{id}/consents/requirements`

What it does: Get caller-specific consent requirements and choices. **Frontend:** Pre-join processing notices, consent screens and in-room withdrawal UI.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/conferences/{id}/consents/{consentType}/withdraw`

What it does: Withdraw caller's processing consent. **Frontend:** Pre-join processing notices, consent screens and in-room withdrawal UI.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,400 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.18 Recordings (4 operations)

Consumer classification: Feature-gated. Primary UI: Recording toolbar, status indicator, post-meeting media library.

`GET /api/v2/conferences/{id}/recordings`

What it does: List conference recordings. **Frontend:** Recording toolbar, status indicator, post-meeting media library.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Distinguish disabled, consent required, provider down, processing and ready.

`POST /api/v2/conferences/{id}/recordings`

What it does: Start idempotent consent-gated recording. **Frontend:** Recording toolbar, status indicator, post-meeting media library.

Request: StartRecordingRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,202,403,404,409,503 **Failure/state:** Distinguish disabled, consent required, provider down, processing and ready.

`GET /api/v2/conferences/{id}/recordings/{recordingId}`

What it does: Read recording status and metadata. **Frontend:** Recording toolbar, status indicator, post-meeting media library.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Distinguish disabled, consent required, provider down, processing and ready.

`POST /api/v2/conferences/{id}/recordings/{recordingId}/stop`

What it does: Stop recording and trigger finalization. **Frontend:** Recording toolbar, status indicator, post-meeting media library.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409 **Failure/state:** Distinguish disabled, consent required, provider down, processing and ready.

A.19 Artifacts & attendance (6 operations)

Consumer classification: Feature-gated. Primary UI: Post-meeting summary, reports, export and secure download.

`GET /api/v2/conferences/{id}/artifacts`

What it does: List conference artifacts. **Frontend:** Post-meeting summary, reports, export and secure download.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Handle unauthorized, expired signed link, missing/deleted object and regeneration.

`POST /api/v2/conferences/{id}/artifacts/reports`

What it does: Generate attendance poll or chat export artifact. **Frontend:** Post-meeting summary, reports, export and secure download.

Request: GenerateReportRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 201,403,503 **Failure/state:** Handle unauthorized, expired signed link, missing/deleted object and regeneration.

`DELETE /api/v2/conferences/{id}/artifacts/{artifactId}`

What it does: Delete authorized artifact. **Frontend:** Post-meeting summary, reports, export and secure download.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,409 **Failure/state:** Handle unauthorized, expired signed link, missing/deleted object and regeneration.

`GET /api/v2/conferences/{id}/artifacts/{artifactId}`

What it does: Get artifact metadata with per-request authorization. **Frontend:** Post-meeting summary, reports, export and secure download.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404 **Failure/state:** Handle unauthorized, expired signed link, missing/deleted object and regeneration.

`POST /api/v2/conferences/{id}/artifacts/{artifactId}/access`

What it does: Create short-lived single-object signed access URL. **Frontend:** Post-meeting summary, reports, export and secure download.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,404,409,410,503 **Failure/state:** Handle unauthorized, expired signed link, missing/deleted object and regeneration.

`GET /api/v2/conferences/{id}/attendance`

What it does: Get attendance intervals and summary. **Frontend:** Post-meeting summary, reports, export and secure download.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token OR platformCredential(meetings.read) **HTTP:** 200,401,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

A.20 Transcription & translation (5 operations)

Consumer classification: Feature-gated. Primary UI: Caption controls, transcript viewer, translation control if enabled.

`POST /api/v2/conferences/{id}/transcription`

What it does: Start allowed live or postmeeting transcript job. **Frontend:** Caption controls, transcript viewer, translation control if enabled.

Request: StartTranscriptionRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 202,403,404,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/transcription/{jobId}/stop`

What it does: Stop transcription job. **Frontend:** Caption controls, transcript viewer, translation control if enabled.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/conferences/{id}/transcripts`

What it does: List conference transcript artifacts. **Frontend:** Caption controls, transcript viewer, translation control if enabled.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`GET /api/v2/conferences/{id}/transcripts/{artifactId}`

What it does: Read paginated finalized transcript segments. **Frontend:** Caption controls, transcript viewer, translation control if enabled.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/conferences/{id}/transcripts/{artifactId}/translations`

What it does: Translate finalized ready transcript. **Frontend:** Caption controls, transcript viewer, translation control if enabled.

Request: TranslateRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.21 Intelligence readiness (1 operations)

Consumer classification: Feature-gated. Primary UI: Gated post-meeting intelligence action and results status.

`POST /api/v2/conferences/{id}/intelligence/requests`

What it does: Request policy-approved AI output from ready source artifact. **Frontend:** Gated post-meeting intelligence action and results status.

Request: IntelligenceRequest **Swagger security:** bearer_auth OR participant_token **HTTP:** 201,403,404,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.22 Webinars & registration (4 operations)

Consumer classification: Feature-gated. Primary UI: Attendee registration form, confirmation and organizer registrations.

`GET /api/v2/meeting-spaces/{id}/registrations`

What it does: List webinar registrations organizer view. **Frontend:** Attendee registration form, confirmation and organizer registrations.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/meeting-spaces/{id}/registrations`

What it does: Register attendee for enabled webinar or town hall. **Frontend:** Attendee registration form, confirmation and organizer registrations.

Request: RegistrationRequest **Swagger security:** anonymous OR bearer_auth **HTTP:** 200,201,400,404,409,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/meeting-spaces/{id}/registrations/{registrationId}`

What it does: Cancel registration and revoke admission token. **Frontend:** Attendee registration form, confirmation and organizer registrations.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403,404 **Failure/state:** Handle stale version/conflict; refetch canonical state before retry.

`GET /api/v2/meeting-spaces/{id}/registrations/{registrationId}`

What it does: Read registration organizer view. **Frontend:** Attendee registration form, confirmation and organizer registrations.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

A.23 Webinars & backstage (2 operations)

Consumer classification: Feature-gated. Primary UI: Panelist/host backstage interface and stage transition.

`DELETE /api/v2/conferences/{id}/backstage`

What it does: Leave backstage and return to stage. **Frontend:** Panelist/host backstage interface and stage transition.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/conferences/{id}/backstage`

What it does: Enter backstage for authorized presenter or host. **Frontend:** Panelist/host backstage interface and stage transition.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.24 Security & meeting profiles (2 operations)

Consumer classification: Feature-gated. Primary UI: Meeting policy settings with effective-policy explanations.

`GET /api/v2/meeting-spaces/{id}/profile-policy`

What it does: Read effective and override meeting policy profile. **Frontend:** Meeting policy settings with effective-policy explanations.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PATCH /api/v2/meeting-spaces/{id}/profile-policy`

What it does: Update versioned policy override within ceilings. **Frontend:** Meeting policy settings with effective-policy explanations.

Request: ProfilePolicyUpdateRequest **Swagger security:** bearer_auth **HTTP:** 200,400,403,409 **Failure/state:** Handle stale version/conflict; refetch canonical state before retry.

A.25 Meeting quality (2 operations)

Consumer classification: Feature-gated. Primary UI: Network health badge, diagnostics and authorized organizer quality view.

`GET /api/v2/conferences/{id}/quality`

What it does: View authorized conference quality aggregates. **Frontend:** Network health badge, diagnostics and authorized organizer quality view.

Request: no-json-body **Swagger security:** bearer_auth OR participant_token **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/conferences/{id}/quality/samples`

What it does: Upload sampled WebRTC network quality telemetry. **Frontend:** Network health badge, diagnostics and authorized organizer quality view.

Request: QualitySampleBatch **Swagger security:** bearer_auth OR participant_token **HTTP:** 202,400,403,404,413,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.26 Legacy account authentication (12 operations)

Consumer classification: Use - account flows. Primary UI: Sign in, sign up, email verification, password recovery and refresh/logout flows.

`POST /api/v1/auth/change-password`

What it does: Change password. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: ChangePasswordRequest **Swagger security:** bearer_auth **HTTP:** 200,400,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/auth/forgot-password`

What it does: Request password reset. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: ForgotPasswordRequest **Swagger security:** not-declared **HTTP:** 200,400,404,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/auth/login`

What it does: Login user. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: LoginRequest **Swagger security:** not-declared **HTTP:** 200,401,403,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/auth/logout`

What it does: Logout user. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v1/auth/oauth/google`

What it does: Initiate Google OAuth. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: no-json-body **Swagger security:** not-declared **HTTP:** 302,500 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`GET /api/v1/auth/oauth/google/callback`

What it does: Google OAuth callback. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: no-json-body **Swagger security:** not-declared **HTTP:** 200,400,401,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v1/auth/refresh`

What it does: Refresh access token. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: RefreshTokenRequest **Swagger security:** not-declared **HTTP:** 200,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/auth/register`

What it does: Register account. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: RegisterRequest **Swagger security:** not-declared **HTTP:** 201,400,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/auth/resend-verification`

What it does: Resend verification email. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: ResendVerificationRequest **Swagger security:** not-declared **HTTP:** 200,400,404,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/auth/reset-password`

What it does: Reset password with token. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: ResetPasswordRequest **Swagger security:** not-declared **HTTP:** 200,400,429 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/auth/verify-email`

What it does: Verify email token. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: VerifyEmailRequest **Swagger security:** not-declared **HTTP:** 200,400,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/auth/verify-reset-token`

What it does: Verify reset token. **Frontend:** Sign in, sign up, email verification, password recovery and refresh/logout flows.

Request: VerifyResetTokenRequest **Swagger security:** not-declared **HTTP:** 200,400,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.27 Account profile & lifecycle (3 operations)

Consumer classification: Use - account flows. Primary UI: Profile/settings screen and account closure confirmation.

`GET /api/v1/users/me`

What it does: Get authenticated user profile. **Frontend:** Profile/settings screen and account closure confirmation.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PATCH /api/v1/users/me`

What it does: Update user profile. **Frontend:** Profile/settings screen and account closure confirmation.

Request: UpdateMeRequest **Swagger security:** bearer_auth **HTTP:** 200,400,401,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/users/me/deactivate`

What it does: Deactivate account. **Frontend:** Profile/settings screen and account closure confirmation.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.28 Legacy meeting compatibility (21 operations)

Consumer classification: Compatibility only. Primary UI: Old-link/old-client fallback only; no new meeting feature built on v1.

`GET /api/v1/meetings`

What it does: Legacy meeting list. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v1/meetings`

What it does: Create legacy meeting. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: CreateMeetingRequest **Swagger security:** bearer_auth **HTTP:** 201,400,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/meetings/instant`

What it does: Create instant legacy meeting. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: InstantMeetingRequest **Swagger security:** bearer_auth **HTTP:** 201,400,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/meetings/join/{meeting_identifier}`

What it does: Join legacy meeting by code. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: JoinMeetingRequest **Swagger security:** not-declared **HTTP:** 200,400,401,403,404,409 **Failure/state:** Map access/passcode/locked/banned/capacity/provider errors to distinct screens; never treat WAITING as joined.

`GET /api/v1/meetings/public/{meeting_identifier}`

What it does: Get legacy public meeting summary. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** not-declared **HTTP:** 200,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`DELETE /api/v1/meetings/{id}`

What it does: Delete legacy scheduled meeting. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 204,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v1/meetings/{id}`

What it does: Get legacy meeting. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PUT /api/v1/meetings/{id}`

What it does: Update legacy meeting. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: UpdateMeetingRequest **Swagger security:** bearer_auth **HTTP:** 200,400,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v1/meetings/{id}/chat`

What it does: Get legacy meeting chat. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v1/meetings/{id}/chat`

What it does: Send legacy chat message. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: SendChatMessageRequest **Swagger security:** bearer_auth **HTTP:** 201,400,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/meetings/{id}/end`

What it does: End legacy meeting. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/meetings/{id}/join`

What it does: Join legacy meeting. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: JoinMeetingRequest **Swagger security:** not-declared **HTTP:** 200,400,401,403,404,409 **Failure/state:** Map access/passcode/locked/banned/capacity/provider errors to distinct screens; never treat WAITING as joined.

`GET /api/v1/meetings/{meeting_id}/participants`

What it does: List legacy meeting participants. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v1/meetings/{meeting_id}/screen-share/start`

What it does: Register legacy screen share start. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/meetings/{meeting_id}/screen-share/stop`

What it does: Register legacy screen share stop. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v1/meetings/{meeting_id}/waiting`

What it does: Get legacy waiting queue. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v1/meetings/{meeting_id}/waiting/{participant_id}/admit`

What it does: Admit legacy waiting participant. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/meetings/{meeting_id}/waiting/{participant_id}/deny`

What it does: Deny legacy waiting participant. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`PATCH /api/v1/participants/{participant_id}/audio`

What it does: Update legacy participant audio status. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: UpdateAudioRequest **Swagger security:** bearer_auth **HTTP:** 200,401,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/participants/{participant_id}/kick`

What it does: Kick legacy participant. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`PATCH /api/v1/participants/{participant_id}/video`

What it does: Update legacy participant video status. **Frontend:** Old-link/old-client fallback only; no new meeting feature built on v1.

Request: UpdateVideoRequest **Swagger security:** bearer_auth **HTTP:** 200,401,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.29 Legacy developer API keys (6 operations)

Consumer classification: Separate developer UI. Primary UI: Developer portal, not the Phase 2 meeting UI.

`GET /api/v1/api-keys`

What it does: List all API keys for authenticated user. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v1/api-keys`

What it does: Create an API key. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: CreateApiKeyRequest **Swagger security:** bearer_auth **HTTP:** 201,400,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v1/api-keys/{id}`

What it does: Revoke API key. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v1/api-keys/{id}`

What it does: Get API key metadata by id. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v1/api/meetings`

What it does: Create meeting via API key. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: ApiCreateMeetingRequest **Swagger security:** api_key_auth **HTTP:** 201,400,401,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v1/api/meetings/{id}/join`

What it does: Join meeting via API key. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: ApiJoinMeetingRequest **Swagger security:** api_key_auth **HTTP:** 200,400,401,403,404,409 **Failure/state:** Map access/passcode/locked/banned/capacity/provider errors to distinct screens; never treat WAITING as joined.

A.30 Legacy webhook administration (4 operations)

Consumer classification: Separate developer UI. Primary UI: Developer portal, not the Phase 2 meeting UI.

`GET /api/v1/webhooks`

What it does: List user webhooks. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v1/webhooks`

What it does: Create user webhook. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: CreateWebhookRequest **Swagger security:** bearer_auth **HTTP:** 201,400,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v1/webhooks/{id}`

What it does: Delete user webhook. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`PATCH /api/v1/webhooks/{id}`

What it does: Update user webhook. **Frontend:** Developer portal, not the Phase 2 meeting UI.

Request: UpdateWebhookRequest **Swagger security:** bearer_auth **HTTP:** 200,400,401,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.31 Unified Messaging identity bridge (Part V) (14 operations)

Consumer classification: Separate Part V integration. Primary UI: Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

`POST /api/v2/messaging/client-sessions/revoke-all`

What it does: Revoke all caller messaging client sessions. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/messaging/client-tokens`

What it does: Issue short-lived device-bound messaging token. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 201,401,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/messaging/devices`

What it does: Register E2EE messaging device. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 201,401,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/messaging/devices/registration-challenges`

What it does: Obtain device key proof challenge. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 201,401,404,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/messaging/devices/{deviceId}/key-packages`

What it does: Publish device one-time key packages. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 201,401,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/messaging/devices/{deviceId}/revoke`

What it does: Revoke caller E2EE messaging device. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 200,401,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/messaging/directory`

What it does: Find eligible organizational messaging colleagues. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/messaging/directory/resolve`

What it does: Resolve messaging principals for colleagues. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 200,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/messaging/me`

What it does: Read caller messaging identity and binding. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,503 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/messaging/principals/names`

What it does: Resolve permitted messaging principal names. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 200,401 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`DELETE /api/v2/messaging/relay/{path}`

What it does: Relay allowlisted delete to unified messaging. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/messaging/relay/{path}`

What it does: Relay allowlisted messaging read. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,404,503 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/messaging/relay/{path}`

What it does: Relay allowlisted messaging create or action. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 200,401,404,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`PUT /api/v2/messaging/relay/{path}`

What it does: Relay allowlisted messaging replacement. **Frontend:** Dedicated E2EE messaging client; not a substitute for conference chat without migration decision.

Request: object **Swagger security:** bearer_auth **HTTP:** 200,401,404,503 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.32 Administrative / migration (not public UI) (11 operations)

Consumer classification: Internal only. Primary UI: Privileged engineering tools; do not expose in public meeting frontend.

`GET /api/v1/admin/logs`

What it does: List session logs (admin only). **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`GET /api/v1/admin/meetings`

What it does: List all meetings (admin only). **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`GET /api/v1/admin/users`

What it does: List all users (admin only). **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,401,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`PATCH /api/v1/admin/users/{id}`

What it does: Update a user (admin only). **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: AdminUpdateUserRequest **Swagger security:** bearer_auth **HTTP:** 200,400,401,403,404 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/admin/chat-migration`

What it does: Inspect chat migration state and evidence. **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403,404 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/admin/chat-migration/parity-runs`

What it does: Run chat migration parity validation. **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/admin/chat-migration/transitions`

What it does: Request audited migration transition. **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: TransitionRequest **Swagger security:** bearer_auth **HTTP:** 200,403,409 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`POST /api/v2/admin/compat/backfill`

What it does: Run idempotent legacy to v2 backfill. **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`GET /api/v2/admin/compat/legacy-meetings/{id}/shadow`

What it does: Compare legacy and v2 meeting state. **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: no-json-body **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`POST /api/v2/admin/organizations`

What it does: Create organization (platform administrator). **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: CreateOrganizationRequest **Swagger security:** bearer_auth **HTTP:** 201,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

`PATCH /api/v2/admin/organizations/{id}`

What it does: Suspend or reactivate organization. **Frontend:** Privileged engineering tools; do not expose in public meeting frontend.

Request: UpdateOrganizationRequest **Swagger security:** bearer_auth **HTTP:** 200,403 **Failure/state:** Show pending and actionable failure; guard duplicate actions; reconcile with canonical GET.

A.33 Infrastructure & probes (2 operations)

Consumer classification: Internal only. Primary UI: Ops health/status UI only; never a participant workflow.

`GET /`

What it does: Root endpoint - Application information. **Frontend:** Ops health/status UI only; never a participant workflow.

Request: no-json-body **Swagger security:** not-declared **HTTP:** 200 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

`GET /health`

What it does: Health check endpoint. **Frontend:** Ops health/status UI only; never a participant workflow.

Request: no-json-body **Swagger security:** not-declared **HTTP:** 200 **Failure/state:** Handle loading, empty, 401/403, 404 and retryable network states.

Appendix B — Swagger cross-domain boundaries and glossary

B.1 Other domains shown in unified Swagger

Swagger family	What it represents	Phase 2 frontend scope
Calling	Independent real-time voice/video/group and enterprise call management	Not part of meeting room implementation; separate product modules.
Numbering / DCNs	Numbers catalog, reservation, assignment, verification, operations	Out of scope.
Privacy Relay	Context-limited masked communication and contacts	Out of scope.
SMS Gateway	Carrier SMS transport, delivery and compliance	Out of scope.
Messaging	Separate conversation, E2EE device/key and message delivery platform	Not a drop-in replacement for conference chat without approved migration.
Meeting Service	Persistent spaces and per-occurrence conferences	This PDR; use source and public-v2 distinctions.
Developer Platform / API Gateway	Developer keys, SDK catalog, public routing, webhooks, usage	Separate developer console project. Not exposed in meeting UI by default.

B.2 Glossary

Term	Meaning
MeetingSpace	Durable reusable meeting configuration, stable code, owner and policies.
ConferenceSession	One specific scheduled/instant/live occurrence of a MeetingSpace.
ConferenceParticipant	One logical authenticated or guest participant in one conference.
ParticipantSession	A device/network/media connection belonging to a participant.
ActorContext	Backend-resolved identity and authorization context, not client-supplied identity.
RTC credential	Short-lived LiveKit/provider token used only for authorized media.
Participant token	Scoped credential allowing a nonregistered actor to access own conference functions.
Realtime ticket	One-time, short-lived authorization for browser WebSocket conference stream.
Outbox event	Committed business change dispatched to event transports with retry support.
Canonical recovery	REST fetch after missed/broken realtime updates to ensure UI truth.
Consent type	Independent RECORDING, TRANSCRIPTION, AI_PROCESSING or DATA_SHARING decision.
Feature gate	Runtime control that determines whether a documented capability is usable.

B.3 Source register and audit notes

[B1] Uploaded: CF_Meet_Phase_2_Backend_Implementation_Specification_v1.1(1).pdf, 29 pp., v1.1, dated 03 Oct 2026. Particularly pp. 2–3 (scope), 7–9 (domain), 11–15 (RTC/collaboration/recording), 18–19 (API/error), 22–28 (security/E2E/closure).

[B2] GitHub: https://github.com/CosmicforgeHealthnet/cfmeet/blob/main/packages/contracts/openapi/meeting-service.json — source-generated v1.2.0, 205 operations as audited.

[B3] GitHub: https://github.com/CosmicforgeHealthnet/cfmeet/blob/main/packages/contracts/bundled/public/meetings/v1.openapi.json — API Gateway-facing 130-v2-operation customer meetings bundle.

[B4] GitHub: https://github.com/CosmicforgeHealthnet/cfmeet/blob/main/services/meeting-service/src/routes/v2.rs — actual route dispatch.

[B5] GitHub: https://github.com/CosmicforgeHealthnet/cfmeet/blob/main/services/meeting-service/docs/PHASE2_MEETING_DOMAIN.md — backend behavioral explanation and known flags.

[B6] GitHub: https://github.com/CosmicforgeHealthnet/cfmeet/blob/main/docs/api/README.md — public/private Swagger split and 25 Oct 2026 production release evidence.

[B7] GitHub: https://github.com/CosmicforgeHealthnet/cfmeet/tree/main/apps/web/src — existing frontend migration input; do not replicate v1/v2 mismatches.

Audit limitation: contracts and GitHub docs were examined, but no fresh live authenticated production/staging API requests, browser RTC certification or all-flag runtime interrogation were executed while preparing this PDR.

END OF IMPLEMENTATION PDR • All 205 Meeting Service HTTP operations appear in Appendix A and the companion endpoint matrix.